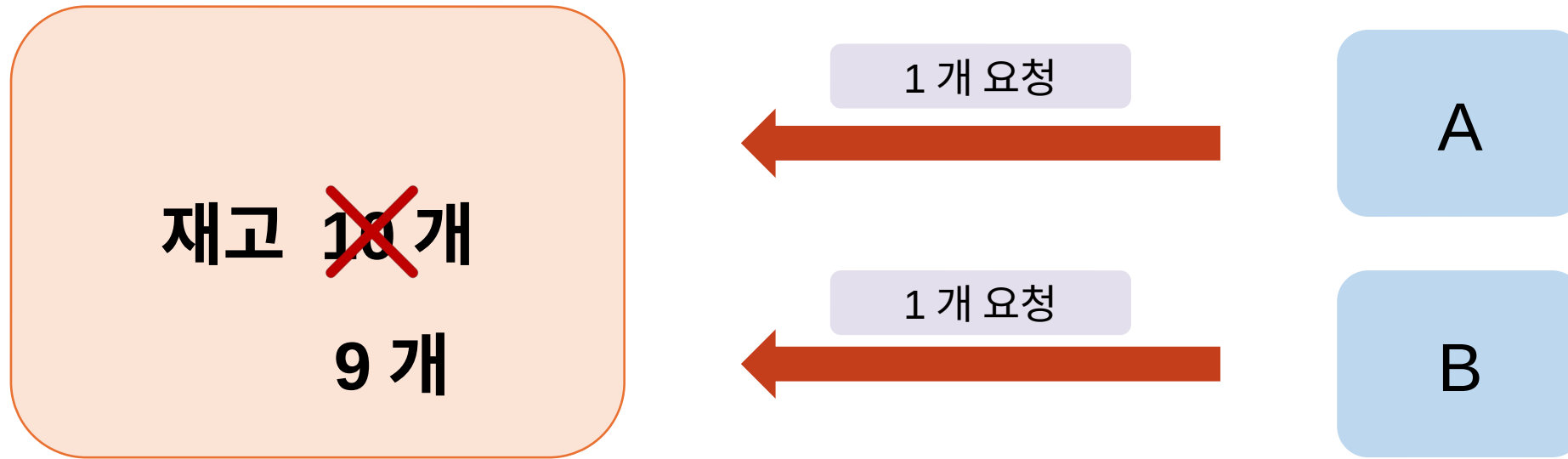


낙관적 락 vs 비관적 락

동시성 제어

- 동일 자원에 대한 다중 요청이 동시에 발생할 때 , 데이터의 일관성과 무결성을 보장하기 위한 제어 메커니즘

동시성이 없다면



A, B 가 동시에 재고를 차감하면 둘 중 하나는 무시된다 .

-> lost update

해결 방법 중 하나 => 락 구현

1. 낙관적 락
2. 비관적 락

낙관적 락

- ➡ 충돌이 적다고 가정하고 시작 .
- ➡ 여러 요청을 일단 동시에 받고 , 나중에 저장하는 시점에 다른 요청이 먼저 수정했는지를 확인한다 .
- ➡ 데이터에 **버전 값**을 두고 조회한 버전과 수정 시점의 버전을 확인한다 .

낙관적 락 (코드예시)

```
1  @Entity
2  public class Product {
3
4      @Id
5      @GeneratedValue(strategy = GenerationType.IDENTITY)
6      private Long id;
7
8      private int stock;
9
10     @Version
11     private Long version;
12
13     public void decreaseStock(int quantity) {
14         if (stock < quantity) {
15             throw new IllegalStateException("재고가 부족합니다.");
16         }
17         stock -= quantity;
18     }
19 }
```

비관적 락

- ➡ 충돌이 발생한다고 가정하고 시작 .
- ➡ 먼저 데이터베이스 락을 획득한 뒤 작업하는 방식
- ➡ 비관적 락은 충돌을 미리 차단하므로 , 대기하는 요청이 늘면 처리량이 떨어질 수 있다 .

비관적 락 (코드예시)



```
1 public interface ProductRepository extends JpaRepository<Product, Long> {  
2  
3     @Lock(LockModeType.PESSIMISTIC_WRITE)  
4     @Query("select p from Product p where p.id = :id")  
5     Optional<Product> findByIdForUpdate(@Param("id") Long id);  
6 }
```

낙관적 락 vs 비관적 락

구분	낙관적 락	비관적 락
기본 가정	충돌이 드물다	충돌이 자주 발생한다
처리 시점	수정할 때 충돌 감지	조회할 때부터 충돌 차단
주요 방법	버전 값 비교	데이터베이스 행 락
다른 요청의 동작	동시에 작업한 뒤 한쪽이 실패할 수 있음	락이 해제될 때까지 대기
장점	락 대기가 적고 조회 성능에 유리	충돌이 잦아도 순차 처리가 명확함
단점	<u>충돌 처리와 재시도 로직이 필요</u>	<u>대기 시간, 타임아웃, 데드락 가능성</u>
적합한 상황	읽기가 많고 충돌이 드문 데이터	같은 데이터에 수정이 집중되는 작업

언제 무엇을 쓸까 ?

< 비관적 략 >

데이터 정합성이 목숨보다 중요할 때 (계좌 이체 , 재고 차감)

충돌이 매우 빈번하게 일어날 때 (선착순 이벤트)

< 낙관적 략 >

성능이 중요하고 충돌이 적을 때 (게시글 수정 , 수강신청)

조회와 수정 사이의 텅이 길 때 (위키 편집 , 관리자 페이지)

참고 자료

<https://www.youtube.com/watch?v=oJrVI6QKzHw>

<https://youtu.be/era8W7q3CeQ?si=daKXi53DOeQaWj52>

